



B.TECH I YEAR II SEM MID-II EXAMINATIONS FEB 2026

WEB APPLICATION DEVELOPMENT-II

(Short Question and Answers)

Unit-I:

1. Define a) Global Scope b) Local Scope c) Block Scope?

Answer:

Global Scope:

- Variables declared outside of any function or block reside in the global scope.
- They are accessible from anywhere in the entire JavaScript program, including within functions and blocks.

Function Scope:

- They are only accessible within the function where they are declared and not outside of it.
- Each function creates its own scope, and variables declared within that function are local to it.

Block Scope:

- Introduced with ES6, variables declared with **let** and **const** inside a block (defined by curly braces `{}` like in if statements, for loops, or even standalone blocks) have block scope.
- These variables are only accessible within the specific block where they are declared.

Example:

```
let globalVar = "I am global"; // Global scope

function exampleFunction()
{
    var functionVar = "I am function-scoped"; // Function scope
    console.log(functionVar);

    if (true)
    {
        let blockVar = "I am block-scoped"; // Block scope
    }
}
```



```
scope
    console.log(blockVar);                                // Accessible
here
    }
}

console.log(globalVar);
exampleFunction();

Output:
'I am global'
'I am function-scoped'
'I am block-scoped'
```

2. What is Hoisting?

Answer:

Hoisting in JavaScript is a behaviour where declarations for functions, variables, classes, and imports are conceptually moved to the top of their containing scope during the compilation phase, before the code is executed.

This allows some elements to be used before they are explicitly declared in the code, though their behaviour varies significantly based on how they are declared.

Example:

```
console.log(a); // using before declaring-- allowed but undefined
var a=10;
console.log(a);
```

output:

```
undefined
10
```

3. What is “this” keyword?

Answer:

The 'this' keyword refers to the object that is currently executing or calling the function. It is a way to access and manipulate the properties and methods of the object that is called the function.



Example:

```
const person = {  
  name: "abc",  
  showName() {  
    console.log(`Hello, my name is ${this.name}`);  
  }  
};  
  
person.showName();  
  
output:  
Hello, my name is abc
```

4. What is Object Destructuring?

Answer:

JavaScript's **destructuring syntax** is a powerful expression that makes it possible to unpack values from arrays or properties from objects into distinct, local variables.

Destructuring is a JavaScript syntax that extracts values from arrays or objects into distinct variables.

Example:

```
let numbers = [10, 20, 30];  
  
let [a, b, c] = numbers;  
  
console.log(a); // 10  
console.log(b); // 20  
console.log(c); // 30
```

5. What is Arrow Function?

Answer:

- Arrow Functions were introduced in ES6 and are commonly used in modern JavaScript.
- Arrow Functions allow a shorter syntax for function expressions. If the function body contains only one statement: You can remove the word function, the curly brackets and the return keyword.

Syntax:



An arrow function uses the `=>` symbol.
It is always written as a function expression.

Example:

```
<!DOCTYPE html>
<html>
<body>

<h1>JavaScript Functions</h1>
<h2>The Arrow Function</h2>
<script>
  // myFunction computes the product of a and b
  const myFunction = (a, b) => a * b;
  let result = myFunction(4, 5);
  console.log("The product is:", result);
</script>
</body>
</html>
```

Output:

The product is: 20

6. what are the types of Destructuring Syntax?

Answer:

Destructuring is a JavaScript syntax that extracts values from arrays or objects into distinct variables.

- **Array Destructuring:** Array destructuring uses square brackets `[]` on the left-hand side of the assignment operator to unpack values based on their position (index) in the array
- **Object Destructuring:** Object destructuring uses curly braces `{}` on the left-hand side to unpack properties into variables with the same name as the property key.
- **Destructuring in Function Parameters:** Function parameter destructuring in JavaScript is a concise syntax that allows you to unwrap values from an object or an array passed as an argument directly into distinct variables within the function's parameter list.
- **Nested Destructuring:** Nested object destructuring in JavaScript allows you to extract properties from objects that are themselves properties of another object. To destructure a nested object, use nested curly braces that match the structure of the object you are accessing.

Unit-II:



7. What is synchronous programming in JavaScript?

Answer:

- **Synchronous programming** refers to a coding execution model where operations are performed one after the other in a strict, linear sequence. Each statement must be completed and an answer returned before the next one can begin.
- By default, JavaScript is a synchronous programming, single-threaded language.

Example:

```
console.log("First task");  
console.log("Second task");  
console.log("Third task");
```

output:

```
First task  
Second task  
Third task
```

8. What is asynchronous programming in JavaScript?

Answer:

Asynchronous JavaScript is a programming approach that enables the non-blocking execution of tasks, allowing concurrent operations, improved responsiveness, and efficient handling of time-consuming operations in web applications.

9. What problem does asynchronous JavaScript solve in single-threaded execution?

Answer:

- **Non-blocking:** The program continues executing subsequent lines of code while waiting for an asynchronous task to finish.
 - **Single-threaded:** JavaScript is naturally single-threaded, meaning it executes one command at a time. It achieves concurrency through its non-blocking behaviour and the event loop.
 - **Event Loop:** This mechanism manages the execution of multiple tasks by moving completed asynchronous callbacks from a queue into the call stack once it is empty.
- There are mainly two types in the Asynchronous programming :



- Callback functions
- Callback Hell and management

10. Define a) Pending state b) Fulfilled state c) Reject state

Answer:

➤ Promises have three states:

- **Pending:** The initial state. The asynchronous operation has not yet completed. The result is initially undefined.
- **Fulfilled (or Resolved):** The operation completed successfully, and a resulting value is available.
- **Rejected:** The operation failed, and an error object (or "reason") is available.

Structure of a Promise

The basic syntax for creating a promise is:

```
const myPromise = new Promise((resolve, reject) => {  
  // Asynchronous operation here  
  if (/* operation successful */) {  
    resolve('Success');  
  } else {  
    reject('Error');  
  }  
});
```

11. What are the three states of a Promise?

Answer:

➤ Promises have three states:

- **Pending:** The initial state. The asynchronous operation has not yet completed. The result is initially undefined.
- **Fulfilled (or Resolved):** The operation completed successfully, and a resulting value is available.
- **Rejected:** The operation failed, and an error object (or "reason") is available.

Structure of a Promise

The basic syntax for creating a promise is:

```
const myPromise = new Promise((resolve, reject) => {  
  // Asynchronous operation here  
  if (/* operation successful */) {  
    resolve('Success');  
  } else {
```



```
reject('Error');  
}  
});
```

12. Define .then() and .catch() method in promise?

Answer:

.then(): .then() is used to handle a fulfilled (resolved) Promise. Runs only when resolve() is called.

Syntax: promise.then(successCallback);

.catch(): .catch() is used to handle a rejected Promise. Runs only when reject() is called.

Syntax: promise.catch(errorCallback);

For example:

```
myPromise  
  .then(result => console.log(result))  
  .catch(error => console.log(error));
```

Unit-III:

13. What is the JavaScript XML and how helps in building React components.?

Answer:

JavaScript XML (JSX) is a syntax extension for JavaScript that allows you to write HTML-like markup directly within your JavaScript files. While it looks like HTML, it is actually a powerful tool that describes what the user interface should look like in a declarative way.

Example:

```
const element = React.createElement (  
  'h1',  
  { className: 'title' },  
  'Hello World!'  
);
```

Writing complex UI trees using React.createElement is tedious and hard to read. JSX allows you to see the structure of your component



at a glance. It bridges the gap between how a UI **looks** and how it **functions**.

Because JSX is compiled into JavaScript objects, React can use a **Virtual DOM** to figure out exactly which parts of the actual page need to change. This makes updates much faster than re-rendering the entire page.

14. What is JSX? Why is it used in React?

Answer:

JavaScript XML (JSX) is a syntax extension for JavaScript that allows you to write HTML-like markup directly within your JavaScript files. While it looks like HTML, it is actually a powerful tool that describes what the user interface should look like in a declarative way.

Example:

```
const element = React.createElement (
  'h1',
  { className: 'title' },
  'Hello World!'
);
```

Writing complex UI trees using `React.createElement` is tedious and hard to read. JSX allows you to see the structure of your component at a glance. It bridges the gap between how a UI **looks** and how it **functions**.

Because JSX is compiled into JavaScript objects, React can use a **Virtual DOM** to figure out exactly which parts of the actual page need to change. This makes updates much faster than re-rendering the entire page.

15. What is React and why is it used for building user interfaces?

Answer:

React is a declarative, component-based JavaScript library used specifically for building user interfaces (UIs). Developed by Meta (formerly Facebook), it has become the most popular tool for front-end development because it changes how we think about building websites.



Instead of writing one giant file of code, you break your UI into small, isolated pieces called **Components**.

- A "Navbar" is a component.
- A "Button" is a component.
- A "Profile Card" is a component.

You can then nest these components inside each other to build complex interfaces. This makes your code **reusable** and much easier to debug.

(Long Question and Answers)

Unit-I:

1. Explain Default binding and Implicit binding of “this” keyword along with example program?

Answer:

- **Default binding of “this” Keyword:**

When a function is called in the global scope, without any object reference, the 'this' keyword refers to the global window object. This is called default binding.

Example:

```
function showThis() {  
    console.log(this);  
}  
  
showThis(); // Window object
```

output:

Window

Implicit binding of “this” Keyword:

In JavaScript, **implicit binding** is a rule that determines the value of the “this” keyword based on the object a function is called on at the time of invocation. This means this automatically refers to the object immediately to the left of the dot (.) when a method is called.

Example:

```
const person = {
```



```
name: "abc",
showName()
{
    console.log("Hello, my name is" + this.name);
}

};

person.showName();
```

output:

Hello, my name is abc

2. What is Error Handling? Explain the Error handling Mechanisms?

Answer:

While writing JavaScript programs, it is very common to make mistakes. These mistakes are called **errors**. An error happens when JavaScript is unable to understand your code or cannot run it properly. When an error occurs, the program may stop suddenly, or it may behave in an unexpected way.

Common reasons for errors include:

- Typing mistakes while writing code
- Writing wrong logic
- Using a variable that has not been created
- Trying to use something before it exists
- Internet or server problems while fetching data

Types of Errors in JavaScript:

- **Syntax Error:** Happens when code grammar is wrong
- **Reference Error:** Happens when something does not exist
- **Type Error:** Happens when a value is used in the wrong way

Error handling is a technique used to **control errors instead of letting them crash the entire program**. Instead of stopping the program suddenly, error handling allows the program to respond in a proper and safe way.



Error handling in JavaScript is primarily managed using the **try...catch...finally** statement. This construct allows you to execute a block of code, catch and manage any errors that occur within it, and run a final block of code regardless of the outcome.

The try...catch block is used to test a block of code for errors.

Syntax:

```
try {  
    .....  
    // Code that might throw an error  
}  
catch (error) {  
    .....  
    // Code to handle the error  
}
```

try block: Contains the code that is monitored for errors or will contain risky code.

catch block:

The catch block runs only if an error occurs inside the try block. It contains the code that runs when an exception is thrown in the try block.

It receives the error object as an argument, which contains information about the error. If no error happens, the catch block is skipped.

Finally statement:

The finally block executes after both the try and catch blocks have completed, regardless of whether an error was thrown or caught. It is often used for cleanup operations, like closing files or releasing resources.

Example:

<script>

```
try{  
    console.log(a);  
}  
catch(error)  
{  
    console.log("something went wrong");  
}
```



</script>

Output:

something went wrong

[try-catch.html:12](#)

3. Explain try, catch, finally block in JavaScript along with example program?

Answer:

While writing JavaScript programs, it is very common to make mistakes. These mistakes are called **errors**. An error happens when JavaScript is unable to understand your code or cannot run it properly. When an error occurs, the program may stop suddenly, or it may behave in an unexpected way.

Common reasons for errors include:

- Typing mistakes while writing code
- Writing wrong logic
- Using a variable that has not been created
- Trying to use something before it exists
- Internet or server problems while fetching data

Types of Errors in JavaScript:

- **Syntax Error:** Happens when code grammar is wrong
- **Reference Error:** Happens when something does not exist
- **Type Error:** Happens when a value is used in the wrong way

Error handling is a technique used to **control errors instead of letting them crash the entire program**. Instead of stopping the program suddenly, error handling allows the program to respond in a proper and safe way.

Error handling in JavaScript is primarily managed using the **try...catch...finally** statement. This construct allows you to execute a block of code, catch and manage any errors that occur within it, and run a final block of code regardless of the outcome.

The try...catch block is used to test a block of code for errors.

Syntax:

```
try {  
    ....  
    // Code that might throw an error
```



```
}  
  catch (error) {  
    .....  
    // Code to handle the error  
  }
```

try block: Contains the code that is monitored for errors or will contain risky code.

catch block:

The catch block runs only if an error occurs inside the try block. It contains the code that runs when an exception is thrown in the try block.

It receives the error object as an argument, which contains information about the error. If no error happens, the catch block is skipped.

Finally statement:

The finally block executes after both the try and catch blocks have completed, regardless of whether an error was thrown or caught. It is often used for cleanup operations, like closing files or releasing resources.

Example:

```
<script>  
  
try{  
  console.log(a);  
}  
catch(error)  
{  
  console.log("something went wrong");  
}  
  
</script>
```

Output:

something went wrong

[try-catch.html](#)



4. Explain Classes in JavaScript in detail along with example?

Answer:

A class Object defines the properties (data) and methods (functions) that all objects created from it will share. **In JavaScript, a class is a blueprint or template for creating objects that share similar properties and methods.**

class keyword: A class is declared using the class keyword, followed by the class name. Class names are typically capitalized by convention.

e.g., class Car {}.

constructor() method: A special method automatically called when a **new** object instance is created using the **new** keyword. It is used to initialize object properties.

new keyword: The mandatory keyword used to create a new instance (object) from a class.

The Constructor Method

The constructor method is a special method:

- It has to have the exact name "constructor"
- It is executed automatically when a new object is created
- It is used to initialize object properties

If you do not define a constructor method, JavaScript will add an empty constructor method.

Class Methods

Class methods are created with the same syntax as object methods.

Use the keyword class to create a class.

Always add a constructor() method.

Then add any number of methods.

Syntax:

```
class ClassName {  
  constructor() { ... }  
  method_1() { ... }  
  method_2() { ... }  
  method_3() { ... }  
}
```

Example program:



```
class Car {
  constructor(name, year) {
    this.name = name;
    this.year = year;
  }

  // method to return car details
  getDetails() {
    return this.name + " - " + this.year;
  }
}

// Creating objects
const myCar1 = new Car("Ford", 2014);
const myCar2 = new Car("Audi", 2019);

// Displaying output in console
console.log("Car 1:", myCar1.getDetails());
console.log("Car 2:", myCar2.getDetails());
```

Output:

```
[ 'Car 1:', 'Ford - 2014' ]
[ 'Car 2:', 'Audi - 2019' ]
```

5. Explain any ten Array Methods in JS along with example?

Answer:

JavaScript provides a rich set of built-in **array methods** to efficiently manipulate, iterate, search, and transform array data. JavaScript provides many **built-in array methods** to work with array elements easily.

Examples:

1. push() – Add element at the end let arr = [1, 2, 3]; arr.push(4); console.log(arr);	2. pop() – Remove last element let arr = [1, 2, 3]; arr.pop(); console.log(arr);	3. unshift() – Add element at the beginning let arr = [2, 3]; arr.unshift(1); console.log(arr);
--	--	---



// Output: [1, 2, 3, 4]	// Output: [1, 2]	// Output: [1, 2, 3]
4. shift() – Remove first element let arr = [1, 2, 3]; arr.shift(); console.log(arr); // Output: [2, 3]	5. length – Get array size let arr = [10, 20, 30]; console.log(arr.length); // Output: 3	6. concat() – Merge arrays let a = [1, 2]; let b = [3, 4]; let result = a.concat(b); console.log(result) ; // Output: [1, 2, 3, 4]
7.slice() – Extract part of array let arr = [10, 20, 30, 40]; let result = arr.slice(1, 3); console.log(result); // Output: [20, 30]	8.splice()– Add/Remove elements let arr = [1, 2, 3, 4]; arr.splice(1, 2); console.log(arr); // Output: [1, 4]	9. forEach() – Loop through array let arr = [1, 2, 3]; arr.forEach(num => console.log(num)); // Output: // 1 // 2 // 3
10. map() – Transform array let arr = [1, 2, 3]; let newArr = arr.map(num => num * 2); console.log(newArr);	11. sort() – Sort array let arr = [3, 1, 4, 2]; arr.sort(); console.log(arr);	12. reverse() – Reverse array let arr = [1, 2, 3]; arr.reverse();



// Output: [2, 4, 6]	// Output: [1, 2, 3, 4]	// Output: [3,2,1]
14.toString() – Convert array to string let arr = [1, 2, 3]; console.log(arr.toString()); // Output: 1,2,3	15.Array.isArray() – Check array let arr = [1, 2]; console.log(Array.isArray(arr)); // Output: true	16.copyWithin() – Copy part inside array let arr = [1, 2, 3, 4]; arr.copyWithin(1, 2); // we copy elements from index 2 to end console.log(arr); // Output: [1, 3, 4, 4]

6. Explain any ten String Methods in JS along with example?

Answer:

JavaScript provides many built-in string methods to manipulate and work with text data. Strings are immutable, meaning these methods return a new string, leaving the original string unchanged.

String Methods and its Examples:

1. length – Get string length let str = "Hello"; console.log(str.length); // Output: 5	2. toUpperCase() – Convert to uppercase let str = "hello"; console.log(str.toUpperCase()); // Output: HELLO	3. toLowerCase() – Convert to lowercase let str = "HELLO"; console.log(str.toLowerCase()); // Output: hello
--	---	---



4. trim() – Remove spaces (start & end) let str = " Hello "; console.log(str.trim()); // Output: Hello	5. charAt() – Get character at index let str = "Hello"; console.log(str.charAt(1)); // Output: e	6. indexOf() – Find position let str = "Hello World"; console.log(str.indexOf("World")); // Output: 6
7. lastIndexOf() – Last occurrence let str = "Hello Hello"; console.log(str.lastIndexOf("Hello")); // Output: 6	8. slice() – Extract part let str = "JavaScript"; console.log(str.slice(4, 10)); // Output: Script	9. substring() – Extract part let str = "JavaScript"; console.log(str.substring(0, 4)); // Output: Java
10. replace() – Replace text let str = "I like JS"; console.log(str.replace("JS", "JavaScript")); // Output: I like JavaScript	11. replaceAll() – Replace all matches let str = "cat cat dog"; console.log(str.replaceAll("cat", "rat")); // Output: rat rat dog	12. includes() – Check existence let str = "Hello World"; console.log(str.includes("World")); // Output: true
13. startsWith() / endsWith() let str = "JavaScript"; console.log(str.startsWith("Java")); console.log(str.endsWith("t"));	14. concat() – Join strings let a = "Hello"; let b = "World"; console.log(a.concat(" ", b));	15. repeat() – Repeat string let str = "Hi "; console.log(str.repeat(3));



<pre>h("Scripts")); // Output: True false</pre>	<pre>// Output: Hello World</pre>	<pre>// Output: Hi Hi Hi</pre>
--	-----------------------------------	--------------------------------

7. Explain Callback Function along with example program?

Answer:

A callback function is a function that is passed as an argument to another function and is executed after the completion of some operations. This mechanism allows JavaScript to perform tasks like reading files, making HTTP requests, or waiting for user input without blocking the execution of the program. This helps ensure a smooth user experience.

Why Use Callback Functions?

JavaScript runs in a single-threaded environment, meaning it can only execute one command at a time. Callback functions help manage asynchronous operations, ensuring that the code continues to run smoothly without waiting for tasks to complete. This approach is crucial for maintaining a responsive and efficient program.

- A function can accept another function as a parameter.
- Callbacks allow one function to call another at a later time.
- A callback function can execute after another function has finished.

Syntax:

```
function outerFunction(data, callback) {
  // Perform some task
  callback(data); // Executing the callback
}
```

Example:



```
function greet(name, callback) {  
  console.log("Hello " + name); 'Hello Husna'  
  callback();  
}  
  
function sayBye() {  
  console.log("Goodbye!"); 'Goodbye!'  
}  
  
greet("Husna", sayBye);
```

LOGS

```
> 'Hello Husna'  
> 'Goodbye!'
```

Why Do We Use Callback Functions?

JavaScript is **asynchronous** (non-blocking). Callbacks help us:

- Handle **delays** (API calls, timers)
- Control **execution order**
- Run code **after a task finishes**.

8. Explain Callback Hell in JavaScript in detail?

Answer:

"Callback hell," also known as the "**Pyramid of Doom**," is a problem in JavaScript that occurs when multiple asynchronous operations are deeply nested using callback functions. This results in code that is difficult to read, debug, and maintain.

What is Callback Hell?

Callback Hell is a situation where multiple nested callbacks make the code difficult to read and maintain.

In JavaScript, callbacks are functions passed as arguments to other functions, to be executed later when an asynchronous operation (like fetching data, reading a file, or a timer) completes.

When multiple asynchronous tasks depend on the result of the previous one, developers historically nested the subsequent callbacks inside the previous ones. This creates a deeply indented, pyramid-like code structure that grows horizontally, making the program's flow difficult to follow.

Callback Hell happens when:

- You have **many nested callbacks**
- Code becomes **hard to read, hard to debug, and hard to maintain**

It looks like a **pyramid / triangle** of functions.



Example:

```
setTimeout(() => {  
  console.log("Step 1"); 'Step 1'  
  
  setTimeout(() => {  
    console.log("Step 2"); 'Step 2'  
  
    setTimeout(() => {  
      console.log("Step 3"); 'Step 3'  
  
      setTimeout(() => {  
        console.log("Step 4"); 'Step 4'  
      }, 1000);  
    }, 1000);  
  }, 1000);  
}, 1000);
```

LOGS

```
> 'Step 1'  
> 'Step 2'  
> 'Step 3'  
> 'Step 4'
```

As seen above, each function is nested within the one before it, forming the characteristic "pyramid" shape.

Why is it a problem?

- **Poor Readability:** The increasing indentation and fragmented logic make the code very hard to read and understand.
- **Difficult Maintenance:** Modifying or adding new features to such a structure is error-prone and time-consuming.
- **Complex Error Handling:** Error management becomes cumbersome, as each nested level must handle potential errors independently, leading to repetitive boilerplate code.
- **Debugging Challenges:** Tracing the flow of execution and locating bugs within deeply nested functions can be a nightmare.

How to AVOID Callback Hell:

Modern JavaScript (ES6+ and Node.js) provides much cleaner ways to handle asynchronous operations and avoid this issue.

- **Promises:** Promises represent the eventual result of an asynchronous operation and allow you to chain operations using `.then()` and `.catch()` methods, which flatten the code structure.
- **async/await:** Built on top of Promises, `async/await` provides a syntax that allows asynchronous code to be written as if it were synchronous, making it the most readable and manageable solution.



- **Modularization/Named Functions:** Breaking the code into smaller, reusable, named functions can reduce nesting even when using the callback pattern, making the code more organized.

Example: Use Named Functions

```
function step1() {  
  console.log("Step 1"); 'Step 1'  
  setTimeout(step2, 1000);  
}  
  
function step2() {  
  console.log("Step 2"); 'Step 2'  
  setTimeout(step3, 1000);  
}  
  
function step3() {  
  console.log("Step 3"); 'Step 3'  
}  
  
setTimeout(step1, 1000);
```

LOGS

```
> 'Step 1'  
> 'Step 2'  
> 'Step 3'
```

9. Explain the concept of “Pyramid of Doom” in detail along with example program?

Answer:

"Callback hell," also known as the "**Pyramid of Doom**," is a problem in JavaScript that occurs when multiple asynchronous operations are deeply nested using callback functions. This results in code that is difficult to read, debug, and maintain.

What is Callback Hell?

Callback Hell is a situation where multiple nested callbacks make the code difficult to read and maintain.

In JavaScript, callbacks are functions passed as arguments to other functions, to be executed later when an asynchronous operation (like fetching data, reading a file, or a timer) completes.

When multiple asynchronous tasks depend on the result of the previous one, developers historically nested the subsequent callbacks inside the previous ones. This creates a deeply indented, pyramid-like code structure that grows horizontally, making the program's flow difficult to follow.

Callback Hell happens when:



- You have **many nested callbacks**
 - Code becomes **hard to read, hard to debug, and hard to maintain**
- It looks like a **pyramid / triangle** of functions.

Example:

```
setTimeout(() => {  
  console.log("Step 1"); 'Step 1'  
  
  setTimeout(() => {  
    console.log("Step 2"); 'Step 2'  
  
    setTimeout(() => {  
      console.log("Step 3"); 'Step 3'  
  
      setTimeout(() => {  
        console.log("Step 4"); 'Step 4'  
      }, 1000);  
    }, 1000);  
  }, 1000);  
}, 1000);
```

LOGS

```
> 'Step 1'  
> 'Step 2'  
> 'Step 3'  
> 'Step 4'
```

As seen above, each function is nested within the one before it, forming the characteristic "pyramid" shape.

Why is it a problem?

- **Poor Readability:** The increasing indentation and fragmented logic make the code very hard to read and understand.
- **Difficult Maintenance:** Modifying or adding new features to such a structure is error-prone and time-consuming.
- **Complex Error Handling:** Error management becomes cumbersome, as each nested level must handle potential errors independently, leading to repetitive boilerplate code.
- **Debugging Challenges:** Tracing the flow of execution and locating bugs within deeply nested functions can be a nightmare.

How to AVOID Callback Hell:

Modern JavaScript (ES6+ and Node.js) provides much cleaner ways to handle asynchronous operations and avoid this issue.

- **Promises:** Promises represent the eventual result of an asynchronous operation and allow you to chain operations using `.then()` and `.catch()` methods, which flatten the code structure.
- **async/await:** Built on top of Promises, `async/await` provides a syntax that allows asynchronous code to be written as if it were synchronous, making it the most readable and manageable solution.
- **Modularization/Named Functions:** Breaking the code into smaller, reusable, named functions can reduce nesting even when using the callback pattern, making the code more organized.



Example: Use Named Functions

```
function step1() {  
    console.log("Step 1"); 'Step 1'  
    setTimeout(step2, 1000);  
}  
  
function step2() {  
    console.log("Step 2"); 'Step 2'  
    setTimeout(step3, 1000);  
}  
  
function step3() {  
    console.log("Step 3"); 'Step 3'  
}  
  
setTimeout(step1, 1000);
```

LOGS

```
> 'Step 1'  
> 'Step 2'  
> 'Step 3'
```

10. Explain timer functions along with example program?

Answer:

JavaScript provides two primary functions, `setTimeout()` and `setInterval()`, to schedule the execution of code asynchronously.

setTimeout(): The `setTimeout()` method executes a function only once after a specified delay in milliseconds.

Syntax: `let timeoutID = setTimeout(function, delay, [arg1, arg2, ...]);`

• Parameters:

- **function:** The function to be executed.
- **delay:** The time, in milliseconds (1000ms = 1 second), to wait before the function is called. The default is 0.
- **arg1, arg2, ... (Optional):** Additional arguments passed directly to the function when it is executed.

Example:



```
function step1() {  
  console.log(" execute step 1 ") ' execute step 1 '  
}  
  
function step2() {  
  console.log(" execute step 2 ") ' execute step 2 '  
}  
  
setTimeout(step1,3000);  
setTimeout(step2,1000);
```

LOGS

```
> ' execute step 2 '  
> ' execute step 1 '
```

setInterval(): The setInterval() method in JavaScript is used to **repeatedly execute a specified function or code at fixed time intervals**. It continues running until explicitly stopped using clearInterval() or the browser window is closed.

Syntax: let intervalID = setInterval(function, delay, [arg1, arg2, ...]);

- **function** (required): The function to be executed repeatedly.
- **delay** (required): The time interval between each execution, specified in milliseconds (1000 milliseconds = 1 second).
- **arg1, arg2, ...** (optional): Additional arguments that are passed to the executed function.
- **intervalID**: The method returns a unique numeric ID that identifies the created timer. This ID is essential for stopping the interval later with clearInterval().

Note: 1 second = 1000 milliseconds.

By using setInterval() method, the output will be printed after 1 sec or 1000ms indefinitely unless you don't stop it or close window.

Note: Since the setInterval() method executes the function infinitely hence there is a method called as clearInterval() to stop the execution of the setInterval().

Example using clearInterval() to stop the execution of the setInterval().



```
function myFunction() {  
  console.log("Executing at regular intervals!"); 'Execut  
}  
  
// store interval id  
let intervalId = setInterval(myFunction, 1000);  
  
// stop the interval after 5 seconds  
setTimeout(() => {  
  clearInterval(intervalId);  
  console.log("Interval stopped!"); 'Interval stopped!'  
, 5000);
```

Output:

LOGS

```
> 'Executing at regular intervals!'  
> 'Executing at regular intervals!'  
> 'Executing at regular intervals!'  
> 'Executing at regular intervals!'  
> 'Interval stopped!'
```

In this example:

- setInterval() returns an ID → intervalId
- clearInterval(intervalId) uses that ID to stop execution
- setTimeout() is used to decide *when* to stop it (after 5 seconds)
- clearInterval() only works with the **interval ID**, not the function name.

11. Explain setTimeout, setInterval functions in detail along with example?

Answer:

JavaScript provides two primary functions, setTimeout() and setInterval(), to schedule the execution of code asynchronously.

setTimeout(): The setTimeout() method executes a function only once after a specified delay in milliseconds.

Syntax: let timeoutID = setTimeout(function, delay, [arg1, arg2, ...]);

• Parameters:

- function: The function to be executed.
- delay: The time, in milliseconds (1000ms = 1 second), to wait before the function is called. The default is 0.



- `arg1, arg2, ...` (Optional): Additional arguments passed directly to the function when it is executed.

Example:

```
function step1() {  
  console.log(" execute step 1 ") ' execute step 1 '  
}  
  
function step2() {  
  console.log(" execute step 2 ") ' execute step 2 '  
}  
  
setTimeout(step1,3000);  
setTimeout(step2,1000);
```

LOGS

```
> ' execute step 2 '  
> ' execute step 1 '
```

setInterval(): The `setInterval()` method in JavaScript is used to **repeatedly execute a specified function or code at fixed time intervals**. It continues running until explicitly stopped using `clearInterval()` or the browser window is closed.

Syntax: `let intervalID = setInterval(function, delay, [arg1, arg2, ...]);`

- **function** (required): The function to be executed repeatedly.
- **delay** (required): The time interval between each execution, specified in milliseconds (1000 milliseconds = 1 second).
- **arg1, arg2, ...** (optional): Additional arguments that are passed to the executed function.
- **intervalID**: The method returns a unique numeric ID that identifies the created timer. This ID is essential for stopping the interval later with `clearInterval()`.

Note: 1 second = 1000 milliseconds.

By using `setInterval()` method, the output will be printed after 1 sec or 1000ms indefinitely unless you don't stop it or close window.

Note: Since the `setInterval()` method executes the function infinitely hence there is a method called as `clearInterval()` to stop the execution of the `setInterval()`.

Example using `clearInterval()` to stop the execution of the `setInterval()`.



```
function myFunction() {  
  console.log("Executing at regular intervals!"); 'Execut  
}  
  
// store interval id  
let intervalId = setInterval(myFunction, 1000);  
  
// stop the interval after 5 seconds  
setTimeout(() => {  
  clearInterval(intervalId);  
  console.log("Interval stopped!"); 'Interval stopped!'  
, 5000);
```

Output:

LOGS

```
> 'Executing at regular intervals!'  
> 'Executing at regular intervals!'  
> 'Executing at regular intervals!'  
> 'Executing at regular intervals!'  
> 'Interval stopped!'
```

In this example:

- `setInterval()` returns an ID → `intervalId`
- `clearInterval(intervalId)` uses that ID to stop execution
- `setTimeout()` is used to decide *when* to stop it (after 5 seconds)
- `clearInterval()` only works with the **interval ID**, not the function name.

12. Explain Promise function and its structure? How do we create promise?

A Promise is a constructor function in JavaScript used to create Promise objects for handling asynchronous operations in JavaScript represents the eventual completion (or failure) of an asynchronous operation and its resulting value.

A Promise is in one of these states:

- **Pending:** The initial state. The asynchronous operation has not yet completed. The result is initially undefined.
- **Fulfilled (or Resolved):** The operation completed successfully, and a resulting value is available.
- **Rejected:** The operation failed, and an error object (or "reason") is available.

Structure of a Promise

The basic syntax for creating a promise is:



```
const myPromise = new Promise((resolve, reject) => {  
  // Asynchronous operation here  
  if (/* operation successful */) {  
    resolve('Success');  
  } else {  
    reject('Error');  
  }  
});
```

Creating Promises

A **Promise** is created using the Promise constructor with the new keyword.

Basic syntax

```
const promise = new Promise((resolve, reject) => {  
  // asynchronous task  
});
```

Example:

```
const myPromise = new Promise((resolve, reject) => {  
  let success = true;  
  
  if (success) {  
    resolve("Task completed successfully");  
  } else {  
    reject("Task failed");  
  }  
});  
myPromise  
  .then(result => console.log(result))  
  .catch(error => console.log(error));
```

Output:

LOGS

```
> Task completed successfully
```

How it works

- Promise is **created**
- success = true → resolve() is called
- .then() receives the value
- Output is printed on the console.



13. Explain the types of components in React in detail?

Answer:

React component is a reusable, self-contained piece of UI. In React, *everything* you see on the screen is built from components—buttons, forms, pages, modals, navbars.

Types Of React Components

1. Functional Components (Modern & Recommended):

Functional components are plain JavaScript functions that return JSX. Since the introduction of **Hooks** (React 16.8), they can manage state, side effects, context, and more—making them the preferred way to write React

2. Class Components (Legacy / Older React):

- **class components** were the only way to use state and lifecycle methods.
- These are ES6 classes that extend `React.Component`.

3. Presentational (UI) Components:

These components focus only on how things look, not how they work.

4. Container (Smart) Components:

Container components handle logic, data fetching, and state, then pass data to presentational components.

5. Controlled Components:

A controlled component is a form element whose value is controlled by React state.

6. Higher-Order Components (HOC):



A Higher-Order Component is a function that takes a component and returns a new component with added behavior.

14. Explain the concept of React in detail?

Answer:

React component is a reusable, self-contained piece of UI. In React, *everything* you see on the screen is built from components—buttons, forms, pages, modals, navbars.

Types Of React Components

1. Functional Components (Modern & Recommended):

Functional components are plain JavaScript functions that return JSX. Since the introduction of **Hooks** (React 16.8), they can manage state, side effects, context, and more—making them the preferred way to write React

2. Class Components (Legacy / Older React):

- **class components** were the only way to use state and lifecycle methods.
- These are ES6 classes that extend `React.Component`.

3. Presentational (UI) Components:

These components focus only on how things look, not how they work.

4. Container (Smart) Components:

Container components handle logic, data fetching, and state, then pass data to presentational components.

5. Controlled Components:

A controlled component is a form element whose value is controlled by React state.

6.Higher-Order Components (HOC):



A Higher-Order Component is a function that takes a component and returns a new component with added behavior.

15. Compare functional components and class components in React with examples.

Answer:

Functional Components	Class Components
A JavaScript function that returns React elements (JSX).	A JavaScript class that extends React.Component and implements a render() method .
Managed using the useState Hook .	Managed using this.state and updated with this.setState() .
Managed using the useEffect Hook (for side effects like data fetching or subscriptions) [1,.	Managed using specific methods like componentDidMount(), componentDidUpdate(), and componentWillUnmount() .
Typically, shorter and easier to read (less boilerplate) .	More verbose due to class structure and this binding .
Not used [1].	Used extensively for accessing props, state, and methods .



Objective (Multiple choice questions)

1. Scope in JavaScript defines the accessibility and visibility of_____
a) variables b) functions c) objects d) all the above [D]
2. Variables declared outside of any function: [A]
a) Global scope b) Local scope c) block scope d) none of the above
3. _____Prevents unpredictable behavior [D]
a) let b) var c) const d) TDZ
4. JavaScript loads the entire function into memory_____ [B]
a) after execution b)before execution c) before compilation d) none
5. _____ variables are only accessible within the specific block [B]
a) global b) block c) functional d) all the above
6. Variables can be declared after it has been used_____hoisting [C]
a) Functional b) global c) variable d) none of the above
7. The value of this is determined at the _____ [D]
a) initialisation b) declaration time c) compiletime d)
runtime
8. _____ function do NOT have their own [B]
a) Bind b)arrow c) apply d)call
9. In synchronous programming JavaScript waits for ___ task to complete
a) 2 b) all c) sequence d) 1 [D]
10. A callback function can execute after another function has _____
a) started b)finished c) initialized d) parallely
[B]
11. _____ happens when the code grammer is wrong [A]
a) Syntax error b)type error c) reference error d)all the above



12. JavaScript is a _____ and synchronous language. [A]
a) single threaded b) multi threaded c) both a and b d) none
13. Synchronous Callbacks Executed immediately during the ____ execution
a) inner function's b) outer function's c) API d) all [B]
14. To manage Callback Hell, we should avoid _____ [D]
a) deep nesting b) write cleaner code c) tasks d) both a and b
15. JavaScript handles asynchronous tasks in the background using _____
a) callbacks hell b) function c) callbacks d) none [C]
16. Syntax for **setTimeout(function, _____);** [A]
a) milliseconds b) seconds c) minutes d) none
17. Promises provide a structured way to handle _____ [C]
a) sync code b) java code c) async code d) all
18. In React, components must return: [C]
a) String b) Number c) JSX d) JSON10.
19. JSX is used to write HTML inside _____ in React? [A]
a) JavaScript b) XML c) HTML5 d) all
20. What is the main advantage of Virtual DOM in React? [C]
a) Faster page reload b) Direct manipulation of real DOM
c) Efficient and faster UI updates d) More memory usage
21. Which tool is used to inspect React components in the browser? [B]
a) Chrome Console b) React Developer Tools
c) Node.js d) Redux DevTools
22. JavaScript can run code in ____ ways: [D]
a) Less than 1 b) Equal to 1 c) Greater than 1 d) two
23. A _____ is a function passed as an argument to another function [A]
a) callback function b) Return function c) function d) all the above
24. Callbacks are commonly used for async operations like _____ [D]
a) timers b) API calls c) file loading d) all
25. Use Promise objects to chain ____ operations using .then() and .catch()



a) synchronous b) array c) asynchronous d) bind [C]

26. React is mainly used for building: [D]

a) Server API's b) node.js c) Web APIs d) user interfaces

27. _____ is used to write HTML inside JavaScript in React? [D]

a) XML b) JavaScript c) HTML5 d) JSX

28. ____scope is determined at the time of writing the code, not when it runs

a) lexical b) functions c) global d) block [A]

29. ____ is JavaScript's default behavior of moving declarations to the top

a) Hoisting b) variables c) functions d) none of the above [A]

30. _____ keyword refers to the object that is currently executing [D]

b) let b) var c) const d) this

Objective- (Fill in the Blanks)

1. _____ reference _____ error happens when something does not exist.

2. _____ Constructor() _____ method automatically called when a new object instance is created using the new keyword.

3. _____ Type _____ Error happens when a value is used in the wrong way.

4. The constructor method has to have the exact name _____ constructor _____.

5. Template Strings allow both single and double quotes inside a _____ string _____.

6. The main goal of error handling is to prevent the program from _____ breaking _____.

7. _____ Class _____ methods are created with the same syntax as object methods.



8. Destructuring is a JavaScript syntax that extracts values from arrays.
9. Template literals are a modern and flexible way to create strings in JavaScript.
10. The spread syntax in JavaScript, denoted by three dots(...).
11. Promises have three states.
12. Consuming a Promise means attaching handlers to it to process its resolution or rejection.
13. The spread syntax(...) is used to expand arrays, objects, or function arguments into individual elements.
14. Promise.all() is a static method that takes an iterable (typically an array) of promises and returns a single promise.
15. The spread operator creates a shallow copy of the person object.
16. Graceful Degradation allows the program to handle failures without crashing the entire script.
17. Async/Await does not block the main thread.
18. The async Keyword: Used to define a function as asynchronous.
19. Await is used ONLY inside async functions.
20. API stands for Application Programming Interface.
21. An API call means JavaScript asks the server for data.
22. A request retrieves data from a specified endpoint.
23. JSX stands for javascript XML JavaScript.
24. React follows a component based architecture.
25. React Developer Tools is available as a browser extension for chrome and firefox.



26. The real DOM is slower compared to the virtual DOM.
27. Promise chaining is a technique in JavaScript used to execute a sequence of asynchronous operations one after another.
28. Flattering code effectively eliminates "Callback Hell" by organizing asynchronous tasks in a linear, readable format.
29. React was developed by facebook(meta).
30. The real DOM is slower compared to the virtual DOM.